Shrinking the etcd Database: Compaction and Defragmentation

Oct 14, 2026 · @Ming

Overview

Shrinking an etcd database takes two steps: compaction, then defragmentation. Running only compaction is the most common mistake, because the file on disk does not get smaller.

Step	What it does	Scope	Shrinks the file?
Compaction	Discards key revisions older than a chosen revision; marks that space free inside the file	Whole cluster, run once	No
Defragmentation	Rewrites the database file and returns free space to the disk	One member at a time	Yes

etcd keeps every version of every key so that `watch` and historical reads work. Without compaction, that history grows until the database hits its size quota.

Step 1: Check the current size and revision

Record the database size before you start so you can confirm the result. The `DB SIZE` column shows the file size of each member.

```
export ETCDCTL_API=3

etcdctl endpoint status --write-out=table
```

Then capture the current revision as a number for the next step:

```
rev=$(etcdctl endpoint status --write-out=json | grep -o '"revision":[0-9]*'
| grep -o '[0-9]*')
echo $rev
```

Take a backup before any maintenance:

```
etcdctl snapshot save backup.db
```

Step 2: Compact to the current revision

Compaction discards all history older than `$rev`. Current values are kept; only older versions of keys are removed.

```
etcdctl compact $rev
```

Compaction applies to the whole cluster, so run it once, not per member. After this step the file size is unchanged until you defragment.

Step 3: Defragment each member

Defragmentation is what actually shrinks the file. It runs per member and blocks reads and writes on that member while it runs, so do one member at a time: followers first, the leader last.

```
etcdctl --endpoints=https://10.0.0.1:2379 defrag
etcdctl --endpoints=https://10.0.0.2:2379 defrag
etcdctl --endpoints=https://10.0.0.3:2379 defrag
```

Avoid `defrag --cluster` on a busy production cluster. It defragments every member back to back and can cause noticeable request delays.

Step 4: Clear the space alarm and verify

If the database reached its size quota (`--quota-backend-bytes`, 2 GB by default), etcd raises a `NOSPACE` alarm and rejects writes with `mvcc: database space exceeded`. Compaction and defragmentation do not clear the alarm on their own.

```
etcdctl alarm list
etcdctl alarm disarm
```

Confirm the result by comparing `DB SIZE` with the value from Step 1:

```
etcdctl endpoint status --write-out=table
```

Running on a kubeadm Kubernetes cluster

kubeadm clusters require TLS for etcd, so every `etcdctl` command needs the certificate flags. Alternatively, run the commands inside the etcd pod with `kubectl -n kube-system exec`.

```
etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint status --write-out=table
```

The Kubernetes API server already compacts etcd every 5 minutes by default (`--etcd-compaction-interval`). On a Kubernetes cluster you usually only need Step 3, unless that automatic compaction has been disabled.

Practical notes

- **Automatic compaction in etcd.** Start etcd with `--auto-compaction-mode=periodic --auto-compaction-retention=1h` to keep one hour of history, or `--auto-compaction-mode=revision --auto-compaction-retention=1000` to keep the last 1,000 revisions.
- **Defragmentation is never automatic.** Schedule it yourself, for example with a cron job during quiet hours, one member at a time.
- **Back up first.** Run `etcdctl snapshot save backup.db` before any maintenance.
- **Persistent growth has a cause.** If the database keeps growing quickly after compaction and defragmentation, look for something writing constantly: a controller updating objects in a loop, many large ConfigMaps, or a flood of Events. Fixing that writer is the real fix.